AAS-Creo-Bridge Documentation

Your Team

Mar 30, 2026

CONTENTS:

1 Introduction **1**
- 1.1 General . . . 1
- 1.2 Topic . . . 1

2 Use Cases **3**

3 Folgender Einleitungstext muss nochmal angepasst werden auf anfängliche 6 cases (Siehe introduction) + wenn möglich bild vom drawio **5**
- 3.1 Engineering . . . 5
  - 3.1.1 1. Scenario and User Role . . . 5
  - 3.1.2 2. Workflow . . . 5
  - 3.1.3 3. Benefits . . . 6
- 3.2 After-Sales . . . 6
  - 3.2.1 1. Scenario and User Role . . . 6
  - 3.2.2 2. Workflow . . . 7
  - 3.2.3 3. Benefits . . . 7

4 Software Utilized **9**

5 Architecture Overview **11**
- 5.1 Core Components . . . 11
  - 5.1.1 `aas_adapter` . . . 11
  - 5.1.2 `aas_creo_bridge` . . . 11
- 5.2 High Level Workflow . . . 11
  - 5.2.1 1. Exporting BOM from CAD to AAS . . . 11
  - 5.2.2 2. Exploring AAS . . . 12
  - 5.2.3 3. Importing Models from AAS to CAD . . . 12

6 Design Decisions **13**
- 6.1 1. Companion Application vs. Integrated Plugin . . . 13
  - 6.1.1 Rationale . . . 13
- 6.2 2. Selection of Creoson vs. Creo Toolkit . . . 14
  - 6.2.1 Rationale . . . 14
- 6.3 3. Selection of Tkinter as a GUI Framework . . . 14
  - 6.3.1 Rationale . . . 14

7 Code Structure and Interactions **17**
- 7.1 Directory Layout . . . 17
- 7.2 Module Responsibilities . . . 17
  - 7.2.1 1. Main Application Controller (`src/aas_creo_bridge/app/`) . . . 17

7.2.2 2. User Interface (`src/aas_creo_bridge/gui/`) . . . 18
7.2.3 3. CAD Adapter (`src/aas_creo_bridge/creo_adapter/`) . . . 18
7.2.4 4. AAS Adapter (`src/aas_adapter/`) . . . 18
7.3 Interaction Flow Examples . . . 18
7.3.1 Generating an AAS from Creo (Export) . . . 18
7.3.2 Extracting a Model from AAS to Creo (Import) . . . 19
7.4 Design Patterns: Function Calls vs. Listeners . . . 19
7.4.1 Direct Function Calls . . . 19
7.4.2 Listeners / Callbacks . . . 19

8 User Guide **21**
8.1 Initial Creoson Setup Guide . . . 21
8.1.1 1. Prerequisites . . . 21
8.1.2 2. Locating and Launching Creoson Setup . . . 21
8.1.3 3. Configuring the Creo Launch Path . . . 21
8.1.4 4. Verifying the Connection . . . 22
8.2 Troubleshooting Guide . . . 22
8.2.1 1. Connection Failures with PTC Creo . . . 22
8.2.2 2. Java Runtime Errors . . . 22
8.2.3 3. AASX Import / Parsing Issues . . . 22
8.3 User Interface Guide . . . 23
8.3.1 Navigation . . . 23
8.3.2 Using the Import View . . . 23
8.3.3 Using the Connection View . . . 24
8.3.4 Exporting a Bill of Materials (BOM) . . . 26
8.3.5 Using the Explorer View . . . 28

9 Installation Guide **31**

10 Getting Started for Developers **33**
10.1 Prerequisites . . . 33
10.2 1) Clone the repository . . . 33
10.3 2) Create and activate a virtual environment . . . 33
10.4 3) Install dependencies . . . 33
10.5 4) Configure IDE . . . 34
10.6 5) Run the application . . . 34
10.7 Testing . . . 34
10.8 Next . . . 34

11 AAS Adapter **35**
11.1 . . . 35
11.1.1 Purpose . . . 35
11.1.2 Featureset . . . 35
11.1.3 Public API . . . 35
11.1.4 Workflow . . . 36
11.1.5 Minimal usage example . . . 36
11.1.6 Known limitations . . . 37
11.1.7 Related Documentation . . . 37
11.2 Importer . . . 37
11.2.1 Overview . . . 37
11.2.2 Key Concepts . . . 38
11.2.3 Main Components . . . 38

11.2.4 Integration Points . . . 39
11.2.5 Error Handling . . . 40
11.2.6 Testing . . . 40
11.2.7 Common Patterns and Gotchas . . . 41
11.2.8 Related Documentation . . . 41
11.3 Registry . . . 41
11.3.1 Overview . . . 41
11.3.2 Key Concepts . . . 42
11.3.3 Main Components . . . 42
11.3.4 Usage Example . . . 43
11.3.5 Integration Points . . . 44
11.3.6 Error Handling . . . 44
11.3.7 Testing . . . 44
11.3.8 Common Patterns and Gotchas . . . 45
11.3.9 Related Documentation . . . 45
11.4 Extractor . . . 45
11.4.1 Overview . . . 45
11.4.2 Key Concepts . . . 46
11.4.3 Main Components . . . 48
11.4.4 MCAD Document Extraction . . . 49
11.4.5 Related: Helpers . . . 49
11.4.6 Error Handling . . . 49
11.4.7 Testing . . . 50
11.4.8 Common Patterns & Gotchas . . . 50
11.4.9 Related Documentation . . . 51

12 App **53**

13 GUI **55**

14 Creo Adapter **57**
14.1 Creo Connection . . . 57
14.1.1 Overview . . . 57
14.1.2 Main Components . . . 57
14.1.3 Behavior . . . 57
14.1.4 Error Handling . . . 58
14.1.5 Testing . . . 58
14.1.6 Common Patterns and Gotchas . . . 58
14.1.7 Related Documentation . . . 58
14.2 Model Import . . . 58
14.2.1 Overview . . . 58
14.2.2 Main Components . . . 59
14.2.3 Behavior . . . 59
14.2.4 Error Handling . . . 59
14.2.5 Testing . . . 59
14.2.6 Common Patterns and Gotchas . . . 60
14.2.7 Related Documentation . . . 60
14.3 Set Parameters . . . 60
14.3.1 Overview . . . 60
14.3.2 Main Components . . . 60
14.3.3 Behavior . . . 61
14.3.4 Error Handling . . . 61

14.3.5 Testing . . . 61
14.3.6 Common Patterns and Gotchas . . . 61
14.3.7 Related Documentation . . . 62
14.4 BOM Export . . . 62
14.4.1 Overview . . . 62
14.4.2 Key Concepts . . . 62
14.4.3 Main Components . . . 62
14.4.4 Error Handling . . . 63
14.4.5 Testing . . . 63
14.4.6 Common Patterns and Gotchas . . . 63
14.4.7 Related Documentation . . . 64
14.5 Types . . . 64
14.5.1 Overview . . . 64
14.5.2 Main Components . . . 64
14.5.3 Related Documentation . . . 66

15 Next Steps **67**
15.1 Open Code Items . . . 67
15.2 Functional Extensions . . . 67

16 Indices and tables **69**

CHAPTER

ONE

INTRODUCTION

1.1 General

In the Mechatronics degree program at Karlsruhe University of Applied Sciences (HKA), students are required to carry out a joint project under the supervision of a professor. Project topics may be proposed by students themselves or offered by professors. The present project was initiated and supervised by Prof. Dr. Michael Hoffmeister.

1.2 Topic

The objective of this project is to establish an initial bridge between Asset Administration Shells (AAS) and CAD systems. An AAS serves as a digital container for all data accumulated throughout a product's lifecycle. This project acts as a pilot to demonstrate how integrating AAS with CAD systems can streamline modern workflows and daily operations. To make the process more tangible, the first step involved defining specific use cases that leverage this connection. Subsequently, the project focused on developing code to implement two of these use cases, which will be showcased as a live demo during the final presentation.

CHAPTER

TWO

USE CASES

CHAPTER

THREE

FOLGENDER EINLEITUNGSTEXT MUSS NOCHMAL ANGEPASST WERDEN AUF ANFÄNGLICHE 6 CASES (SIEHE INTRODUCTION) + WENN MÖGLICH BILD VOM DRAWIO

At the beginning of this project, several functional areas of a hypothetical industrial company were outlined and discussed with respect to how the AAS could be applied in each context. Subsequently, the scope was narrowed down to two specific use cases: Engineering and After-Sales.

3.1 Engineering

3.1.1 1. Scenario and User Role

Robert Redford is a design engineer at an actuator manufacturer. He is responsible for developing a new vacuum gripper. During the development phase, he must ensure that all relevant requirements from the Machinery Directive, applicable safety standards, and documentation requirements are fulfilled. These requirements are digitally stored as properties and documents within the gripper's AAS.

Robert is looking for a suitable vacuum generator (e.g., suction module) for a new gripper.

3.1.2 2. Workflow

Component Selection

Robert checks the requirements stored in the AAS of the vacuum gripper (e.g., safety standards, technical limits). He searches the supplier's catalog for a suitable vacuum generator and downloads its AAS.

Integration of the AAS

The AAS of the vacuum generator is imported into the AAS of the vacuum gripper. In the AAS editor or directly within the CAD system, the connections between the gripper and the vacuum generator (e.g., pneumatic, electrical, mechanical) are defined. The geometry of the vacuum generator is automatically placed within the CAD model of the gripper.

Bill of Material (BOM)

After integration, Robert can generate the updated BOM directly in the CAD system. The BOM is automatically updated and stored as a submodel within the AAS and now includes the new vacuum generator along with all associated data.

3.1.3 3. Benefits

Seamless Integration

Components from different manufacturers can be seamlessly integrated because the AAS serves as a standardized exchange format.

Automation

Component integration and BOM updates are performed automatically.

Regulatory Compliance

All requirements from the Machinery Directive and applicable safety standards are digitally documented and can be verified at any time.

Lifecycle Transparency

All technical, safety-related, and sustainability-related information is centrally managed within the AAS. This creates full transparency across the entire product lifecycle, from design and operation to maintenance, optimization, and spare parts management.

3.2 After-Sales

3.2.1 1. Scenario and User Role

Once the vacuum gripper is in operation, the after-sales phase begins, including: maintenance, repair, and spare parts management. Paul Newman, a service technician at the same company, who supports customers in the field. When a malfunction is reported, such as a sudden loss of vacuum pressure, he uses the digital product data stored in the AAS to diagnose and resolve the issue efficiently.

The CAD assemblies created during development serve as his central reference. Using these models, together with derived views such as exploded views and assembly diagrams, Paul can perform maintenance tasks accurately and reliably.

3.2.2 2. Workflow

Preparation

A customer reports a sudden loss of vacuum pressure during operation for example. Paul opens the relevant assembly in Creo, where the corresponding disassembly instructions from the AAS are automatically displayed in a second window.

Identification

Paul selects the affected component directly within the CAD model. All relevant metadata, such as part numbers and material specifications, is automatically retrieved from the AAS. This helps him quickly identify the likely cause of the pressure loss.

Repair

Paul follows the step-by-step repair instructions provided through the AAS. These instructions include required tools, safety considerations, torque values, and all other details needed to replace or repair the faulty component correctly.

Feedback

If the same issue occurs repeatedly across multiple service cases, Paul documents it in the after-sales system. This feedback is routed through the AAS back to the engineering department, enabling targeted design improvements for future product iterations.

3.2.3 3. Benefits

Increased Efficiency

Service technicians receive all relevant information directly within the CAD system, reducing both repair time and the likelihood of errors.

Traceability

Every service case is clearly documented and can be traced back to the affected assembly and the specific spare part involved.

Feedback Loop

Insights gained during service operations are systematically fed back to the engineering department, enabling targeted product improvements.

Regulatory Compliance

All relevant information on hazardous substances, recyclability, and part interchangeability is available digitally and supports compliance with regulatory requirements.

Automation

The integration of CAD and AAS enables largely automated provisioning and updating of service-related information.

CHAPTER

FOUR

SOFTWARE UTILIZED

The project combines proprietary CAD software with open source tooling and libraries. The main components and their licenses are summarized below.

- Creo 12 (PTC): Proprietary CAD system used under a student license. Use is governed by the PTC software license agreement; redistribution is not permitted outside that agreement.
- PyCharm: Development environment by JetBrains. The Community Edition terms cover Apache 2.0 open source code plus bundled proprietary plugins; use of free editions for commercial development is permitted. Pro features require a paid subscription after the trial period.
- Python: PSF License (v2). Permissive use, modification, and redistribution, provided the PSF license text and copyright notice are retained in distributions.
- pytest: MIT License. Permissive use, modification, and redistribution, provided the copyright and license notice are included in distributions.
- Creoson Server (Simplified Logic): MIT License. Permissive use, modification, and redistribution, provided the copyright and license notice are included in distributions.
- Creopyson (Python client for Creoson): MIT License. Permissive use, modification, and redistribution, provided the copyright and license notice are included in distributions.
- Eclipse BaSyx Python SDK (`basyx_python_sdk`): MIT License. Permissive use, modification, and redistribution, provided the copyright and license notice are included in distributions.
- PyECMA376-2 (`pyecma376_2`): Apache License 2.0. Permissive use, modification, and redistribution with required license text and NOTICE retention for attribution where applicable.

CHAPTER

FIVE

ARCHITECTURE OVERVIEW

This project is structured as a bridge between CAD Systems (currently PTC Creo Parametric) and the Asset Administration Shell (AAS). It uses an adapter pattern to separate concerns and provide a clean interface for interacting with the CAD system and the AAS.

5.1 Core Components

5.1.1 `aas_adapter`

This module provides the necessary interfaces for generating, importing, and manipulating AAS structures building upon the `basyx.aas` library.

5.1.2 `aas_creo_bridge`

The central application logic that unifies the CAD processes and user interfaces.

1. **App**: Main context, configuration, and state management.
2. **GUI**: A Tkinter-based user interface enabling visual interaction with the AAS repositories and the CAD system.
3. **Creo_adapter**: Connects to the Creo API. Uses JLink and `creopyson` behind the scenes to extract Bills of Materials (BOM), inject properties, and import model files.

5.2 High Level Workflow

The application supports several key workflows:

5.2.1 1. Exporting BOM from CAD to AAS

1. The Bridge connects to a running Creo session via Creoson / Jink.
2. The user initiates a BOM extraction from the **GUI**.
3. **Creo_adapter** reads the hierarchical structure and parts information from Creo.
4. The extracted JSON/Dict data is provided to the **aas_adapter**.
5. The Bridge logic generates or updates AAS models (e.g. Submodels like "BillOfMaterial" or "DigitalNameplate").

6. The updated AASX file or repository is saved.

5.2.2 2. Exploring AAS

1. The user opens an AASX package or connects to an AAS repository via the **GUI**.
2. The **aas_adapter** parses and loads the Asset Administration Shells, Submodels, and property data.
3. The application displays the hierarchy, allowing the user to inspect properties, relationships, and associated files.

5.2.3 3. Importing Models from AAS to CAD

1. While exploring an AAS, the user identifies an attached CAD model file.
2. The user triggers an import action via the **GUI**.
3. The **aas_adapter** extracts the physical file from the AAS environment.
4. The **Creo_adapter** sends commands to Creo to load and open the model in the active session.

CHAPTER

SIX

DESIGN DECISIONS

This document outlines the core architectural and technological decisions made during the development of the AAS-Creo-Bridge. It explains the rationale behind the chosen approaches, frameworks, and tools.

6.1 1. Companion Application vs. Integrated Plugin

The AAS-Creo-Bridge is designed as a standalone "companion" application that runs alongside PTC Creo Parametric, rather than as an embedded plugin or integrated toolkit extension running directly inside the Creo process.

6.1.1 Rationale

- **Separation of Concerns and Stability:** Running as a separate process ensures that any potential crashes, memory leaks, or heavy computational tasks originating from the AAS manipulation (such as parsing large AASX bundles) do not affect the stability of the Creo CAD session.
- **Technology Stack Flexibility:** Creo plugins are typically written in C/C++ (Creo Toolkit) or Java (J-Link). By using a companion app architecture that communicates via an API (Creoson), we were able to write the entire application logic, data transformation, and GUI in Python. Python offers superior libraries for data manipulation, rapid prototyping, and interacting with the Asset Administration Shell (via `basyx.aas`).
- **Decoupled Updates:** The companion app can be updated, restarted, and deployed independently of the CAD system. There is no need to restart Creo or manage complex internal DLL/JAR loading mechanisms during development or deployment.
- **Cross-Domain Interaction:** A standalone application acts as a clear middle-man. It visualizes the AAS domain separately from the CAD domain, making it easier for users to understand the mapping and bridge between the two environments without cluttering the native Creo user interface.

6.2 2. Selection of Creoson vs. Creo Toolkit

To bridge the gap between our Python application and PTC Creo, we chose to use **Creoson** (accessed via the `creopyson` Python wrapper) instead of the native **Creo Toolkit** (C/C++) or **J-Link** (Java) APIs directly.

6.2.1 Rationale

- **Language Barrier:** The native Creo APIs are heavily tied to C/C++ and Java. Calling native C++ Creo Toolkit functions from Python requires complex bindings (such as ctypes, SWIG, or Pybind11) which are difficult to maintain and platform-dependent.
- **Microserver Architecture:** Creoson acts as a microserver that exposes Creo's internal J-Link API over standard JSON-RPC via HTTP. This modernizes the integration, turning complex CAD API calls into simple, language-agnostic web requests.
- **Ease of Use & Community:** Creoson and its Python wrapper, `creopyson`, abstract away many of the low-level difficulties of communicating with Creo. Extracting a Bill of Materials (BOM) or injecting a parameter becomes a simple Python function call returning a dictionary, rather than requiring verbose boilerplate code for memory management and Creo session handling.
- **Trade-offs:** While Creoson is incredibly easy to use, it does introduce a slight performance overhead due to the HTTP/JSON serialization step compared to direct in-memory C++ Toolkit calls. However, for metadata extraction, BOM parsing, and parameter injection (which are the core use cases for this bridge), this overhead is negligible and heavily outweighed by the reduction in development time and complexity.

6.3 3. Selection of Tkinter as a GUI Framework

The graphical user interface for the AAS-Creo-Bridge is built using **Tkinter**, Python's native GUI toolkit.

6.3.1 Rationale

- **Zero Dependencies & Built-in:** Tkinter is included with standard Python installations on Windows. This eliminates the need to bundle large third-party GUI frameworks, keeping the application's footprint smaller and simplifying the build and deployment process (e.g., when packaging with PyInstaller).
- **Sufficient Feature Set:** The application primarily requires standard UI components: tree views (for displaying the CAD BOM and AAS hierarchy), property tables, input forms, and progress bars. Tkinter, specifically with the `ttk` (Themed Tkinter) extensions, provides all these widgets natively.
- **Simplicity and Speed of Development:** Tkinter is straightforward to implement and well-documented. For an application focused heavily on backend transformations (CAD to AAS) rather than highly custom animations or complex graphical rendering, Tkinter allowed for rapid development of a functional and clean interface.
- **Trade-offs:** Tkinter's default look and feel can appear slightly dated out-of-the-box compared to modern frameworks like PyQt/PySide or web-based electron apps. However, using `ttk` themes

helps mitigate this, providing a native OS appearance that is more than adequate for a professional engineering utility.

CHAPTER

SEVEN

CODE STRUCTURE AND INTERACTIONS

This document provides a detailed overview of the directory structure and the internal modules of the AAS-Creo-Bridge. It explains how the different parts of the code interact to provide the core functionalities.

7.1 Directory Layout

```
AAS-Creo-Bridge/
├── docs/                   # Documentation (Sphinx, Architecture, Guides)
├── src/                    # Main source code directory
│   ├── aas_adapter/        # Module for interacting with Asset Administration␣
↪Shells
│   └── aas_creo_bridge/    # Application core, GUI, and CAD (Creo) adapters
├── tests/                  # Unit and Integration test suite (pytest)
├── creoson/                # Creoson microserver binaries and configuration
├── scripts/                # Build and utility scripts
└── root level files        # configurations (setup.py, pytest.ini,␣
↪requirements.txt, etc.)
```

7.2 Module Responsibilities

The application uses an adapter pattern to separate presentation, application state, CAD interaction, and AAS manipulation.

7.2.1 1. Main Application Controller (`src/aas_creo_bridge/app/`)

- Acts as the central orchestrator and state manager.
- Manages configurations, user preferences, and the logging infrastructure.
- Serves as the middle layer connecting user actions from the GUI to the backend adapters.

7.2.2 2. User Interface (`src/aas_creo_bridge/gui/`)

- A desktop interface built using Tkinter.
- Captures user input (e.g., connecting to Creo, opening an AAS file, triggering an export or import).
- Translates UI events into application commands sent to the Application Controller.
- Receives data back to render trees, tables, and property views.

7.2.3 3. CAD Adapter (`src/aas_creo_bridge/creo_adapter/`)

- Interfaces strictly with PTC Creo Parametric.
- Communicates predominantly via `creopyson` mapping to the local `creoson` server.
- Performs all CAD-specific operations:
  - Extracting Assembly Trees & Bill of Materials (BOM).
  - Injecting/Reading part parameters.
  - Sending open/load commands for downloaded CAD models.
- Returns CAD-agnostic Python dictionaries and lists.

7.2.4 4. AAS Adapter (`src/aas_adapter/`)

- Built heavily on the Eclipse BaSyx (`basyx.aas`) library.
- Performs all AAS-specific operations:
  - Reading, parsing, and writing `.aasx` bundles.
  - Generating standard Submodels (e.g., Hierarchical structures, Digital Nameplate).
  - Extracting physical payload files (like `.prt` or `.asm` files) contained within the AAS structure.

7.3 Interaction Flow Examples

7.3.1 Generating an AAS from Creo (Export)

1. **Trigger**: The User clicks "Export to AAS" in the **GUI**.
2. **Command**: The GUI calls the corresponding export function on the **Main Application**.
3. **Data Retrieval**: The Main Application requests the assembly structure from the **Creo_adapter**. The adapter communicates with Creo, parses the active assembly, and returns a sanitized data structure.
4. **Transformation**: The Main Application delegates the standardized CAD data to the **AAS Adapter**.
5. **AAS Construction**: The AAS Adapter models the components as `AssetAdministrationShell` objects and maps properties into standardized `Submodel` formats.

6. **Storage**: The AAS Adapter bundles the environment into an `.aasx` file and saves it to disk.
7. **Feedback**: The Main Application receives a success response and prompts the GUI to update itself.

7.3.2 Extracting a Model from AAS to Creo (Import)

1. **Trigger**: The User selects a CAD file inside an AAS package via the **GUI** and clicks "Load into Creo".
2. **Command**: The GUI passes the file reference to the **Main Application**.
3. **Extraction**: The Main Application requests the **AAS Adapter** to extract the specific payload file from the AASX package to a temporary local folder.
4. **Execution**: The Main Application sends the temporary file path to the **Creo_adapter** with an instruction to "Open".
5. **CAD Action**: The Creo_adapter sends the appropriate RPC command to Creoson to load the part into the active Creo session.

7.4 Design Patterns: Function Calls vs. Listeners

The application relies on two primary communication patterns between its modules: **Direct Function Calls** and **Event Listeners (Callbacks)**.

7.4.1 Direct Function Calls

- **Usage**: Used for synchronous operations where the caller depends on the immediate return of data or status.
- **Context**: Communication between the Main Application Controller and the backend adapters (AAS Adapter and Creo Adapter) predominantly uses direct function calls. This ensures operations like parsing data, modifying data structures, or making fast local API calls (like to the local Creoson server) occur in a predictable and sequential manner.
- **Example**: `creo_adapter.get_assembly_tree()` directly returns the extracted BOM structure.

7.4.2 Listeners / Callbacks

- **Usage**: Used for asynchronous status updates and decoupling the GUI from the backend application logic.
- **Context**: The GUI often registers listener functions with the Main Application Controller. When long-running tasks occur (e.g., parsing a massive AASX file or performing a complex CAD extraction), the backend invokes these listeners to update progress bars, log outputs, or notify the user interface that new data is ready to be rendered.
- **Example**: The GUI registers an `on_progress_update` callback. As the AAS Adapter iterates through submodels, it triggers this callback, allowing the Tkinter main loop to safely update the visible progress bar without freezing.

CHAPTER

EIGHT

USER GUIDE

8.1 Initial Creoson Setup Guide

The AAS-Creo-Bridge relies on **Creoson** to communicate with PTC Creo Parametric. When starting the application or attempting a connection for the first time, you may need to configure Creoson to ensure it can successfully interact with your local Creo environment.

8.1.1 1. Prerequisites

- **PTC Creo Parametric 12** must be installed on your machine.
- **J-Link** must be enabled in your Creo installation. (Note: This is an optional component during the PTC Creo installation process if it was not installed by default).

8.1.2 2. Locating and Launching Creoson Setup

If the Bridge cannot establish a connection automatically, the Creoson Setup utility is launched. There is a chance that the Creoson Setup gets stuck in a loop during startup. If this happens:

1. Kill Creoson Setup from the task manager.
2. Navigate to the `creoson` directory within your application or workspace folder.
3. Delete the `setvars.bat` file.
4. Launch the AAS-Creo-Bridge again.

8.1.3 3. Configuring the Creo Launch Path

In the Creoson Setup utility, you have to define the location of Creo Parametric so that Creoson knows how to connect to the session.

1. Set the Path to Creo Parametric:
   - *Example Path*: `C:\Program Files\PTC\Creo 12.4.0.0\Common Files`
2. Keep the default Setting for the Port
3. Click `Start CREOSON` and close the setup utility.

8.1.4 4. Verifying the Connection

1. Launch the AAS-Creo-Bridge application.
2. In the side navigation, go to the **Connection View**.
3. On the right side models that are currently open in Creo should appear.

8.2 Troubleshooting Guide

When integrating different systems like CAD software and AAS instances, several common issues can arise. This guide provides solutions to frequently encountered problems.

8.2.1 1. Connection Failures with PTC Creo

Symptom: The Bridge cannot connect to Creoson

- **Is Creo running?** Ensure a valid PTC Creo session is active.
- **J-Link Missing:** Ensure that J-Link is installed and registered with your PTC Creo installation. If J-Link is absent, Creoson cannot communicate with Creo. Re-run the PTC Creo installer and ensure "J-Link" is checked.
- **Incorrect Startup Path:** Ensure your `setvars.bat` points to the correct installation folder of `PTC Creo Parametric`. Refer to the *Creoson Setup Guide*.

8.2.2 2. Java Runtime Errors

Symptom: Creoson crashes or fails to start with Java errors.

- **Missing JRE:** Ensure you have Java 21+ installed on your system or that the embedded JRE option was used during setup.
- **Architecture Mismatch:** Ensure you are using the 64-bit version of Java if your operating system and Creo are 64-bit.

8.2.3 3. AASX Import / Parsing Issues

Symptom: Importing an AASX file fails or shows empty contents.

- **Corrupted File:** Try opening the AASX file in twinfix to verify its integrity.
- **Unsupported Specification Version:** The Bridge relies on `basyx.aas`. Ensure the `.aasx` package was generated adhering to a supported version of the Asset Administration Shell specifications (V3.0+).

8.3 User Interface Guide

The application interface is divided into functional views accessible via a side navigation pane.

8.3.1 Navigation

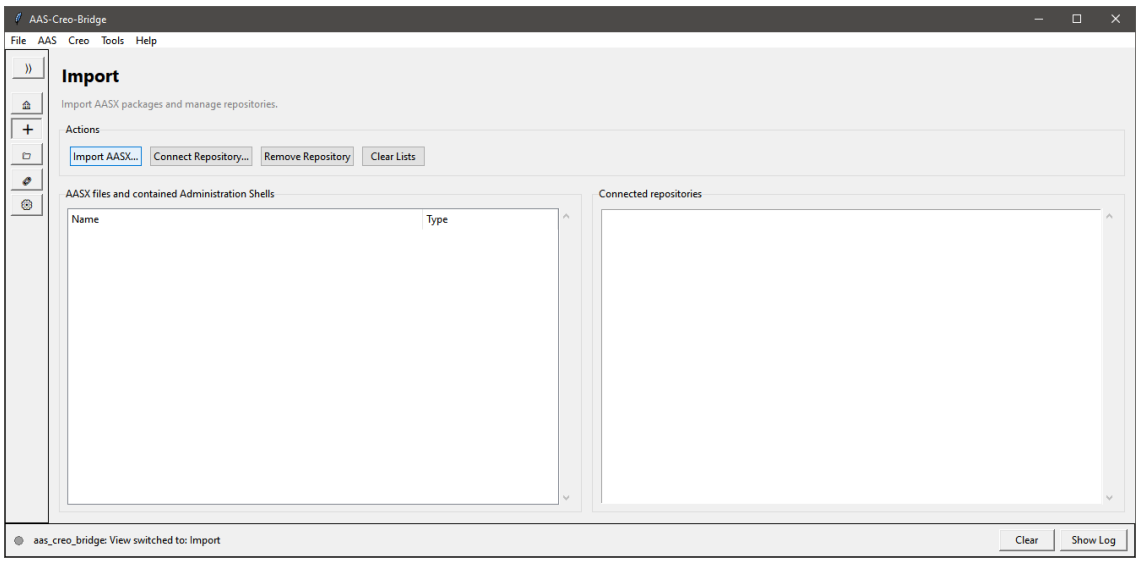
The navigation pane allows switching between the different workflows:

- **Import View**: Allows opening locally stored AASX files.
- **Explorer View**: Enables inspection of the AAS structures, submodels, and properties.
- **Connection View**: Handles the connection to the CAD system and controls the extraction.

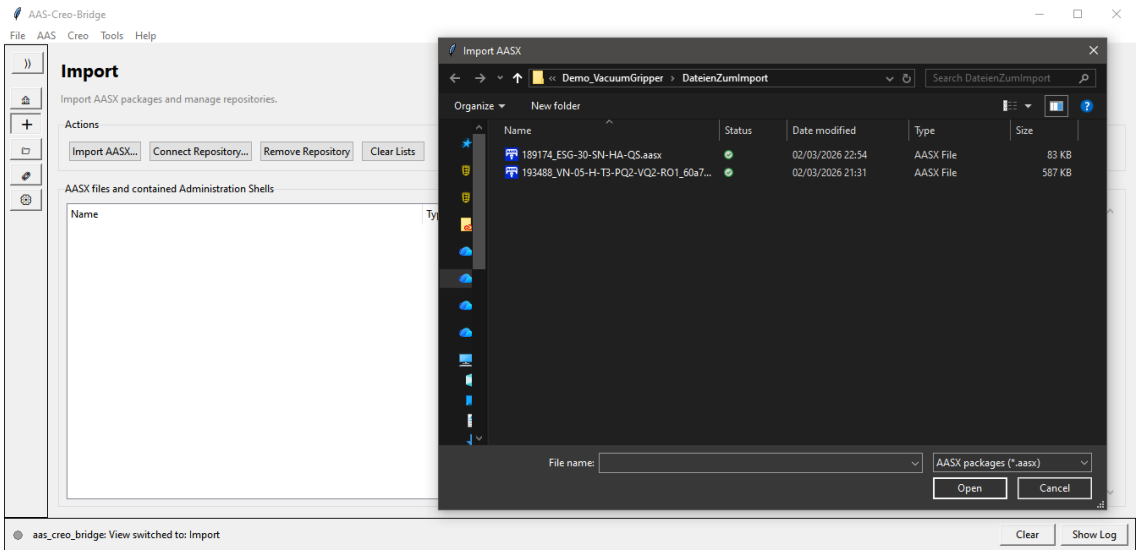
8.3.2 Using the Import View

To import an existing Asset Administration Shell into the application, follow these steps:

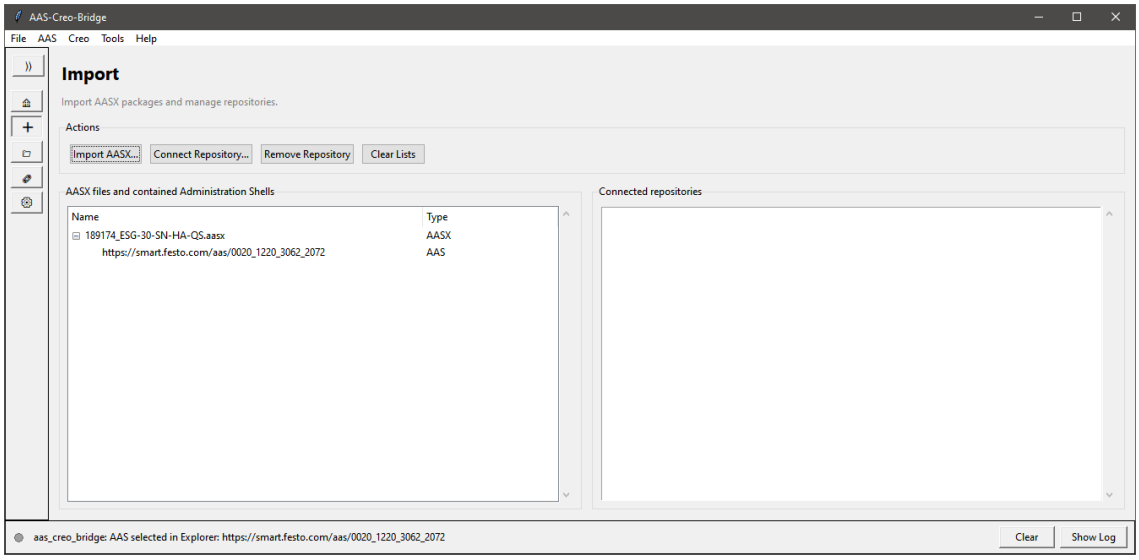
1. **Navigate to the Import View**: Look at the side navigation pane on the left side of the application window. Click on the plus icon to open the **Import** tab.



2. **Open the File Dialog**: Once in the Import View, locate the "Actions" section and click the **Import AASX…** button.
3. **Select your AASX File**: A standard file dialog will open. Browse your local file system, select the desired `.aasx` package, and click **Open**.



4. **Verify the Import**: The imported Asset Administration Shells will now populate in the tree view on the left pane. You can expand the root node to inspect the individual shells contained within the package.

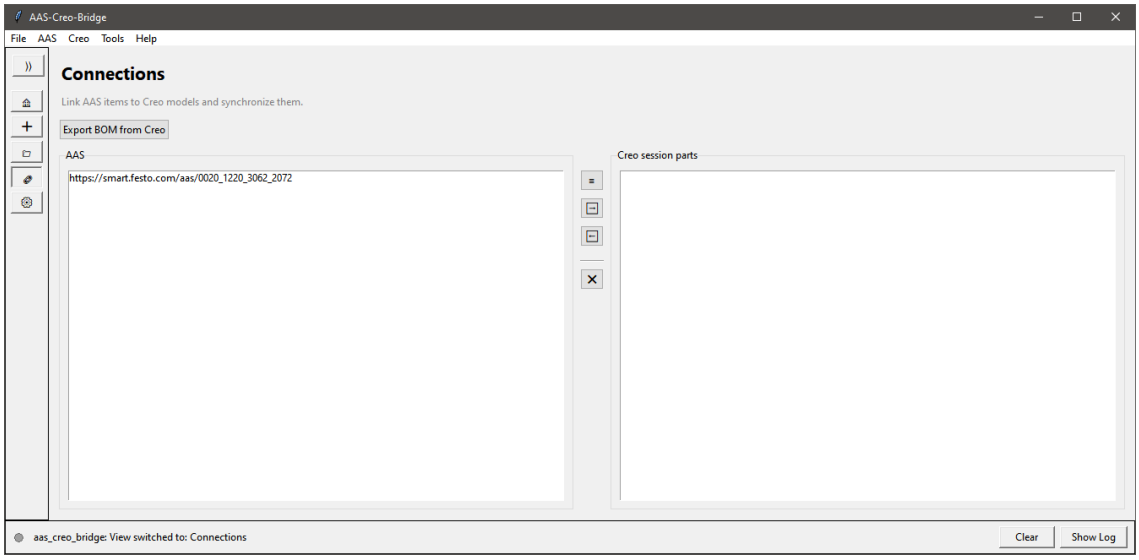


Connecting to a repository is not supported yet.

8.3.3 Using the Connection View

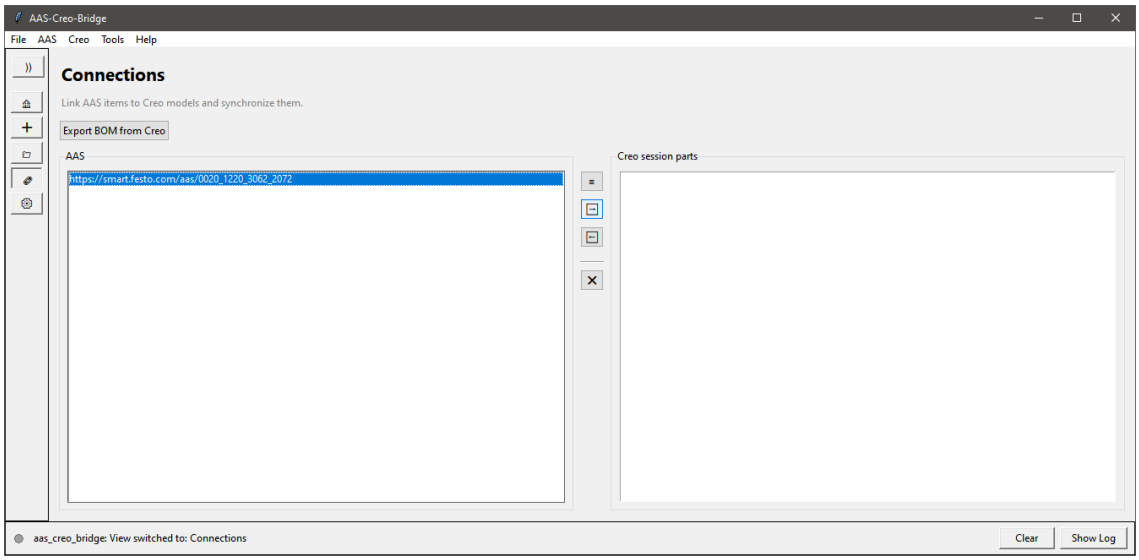
The Connection View is where you synchronize the active Creo session with your imported Asset Administration Shells.

1. **Navigate to the Connection View**: Look at the side navigation pane on the left side of the application window. Click on the chain link icon to open **Connection** tab.

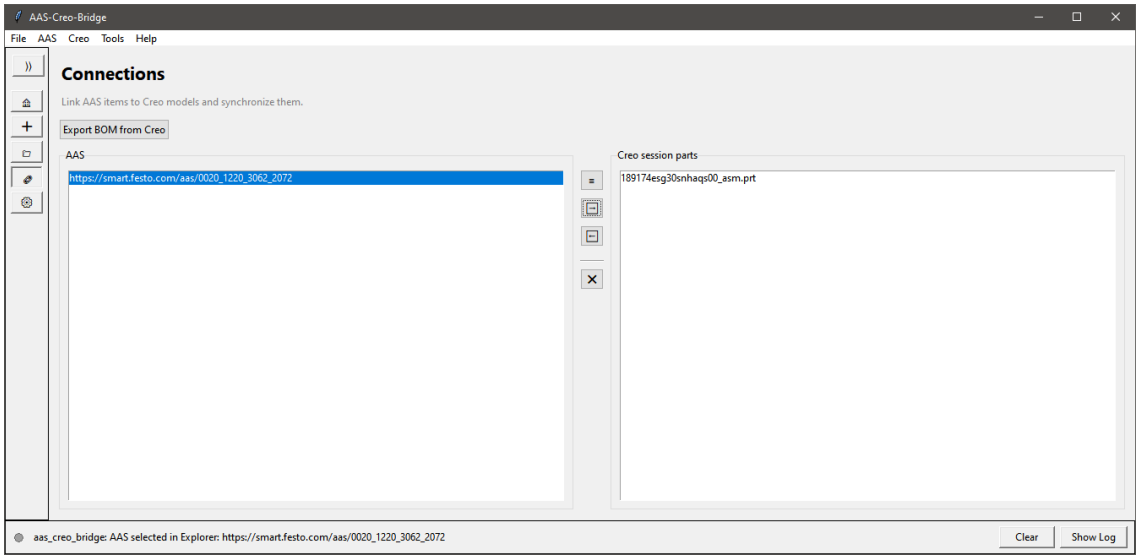


2. **Establish Connection to Creo**: Ensure PTC Creo is running with the correct model open (or an empty session if you are importing). The bridge will automatically attempt to connect to the Creoson background service.

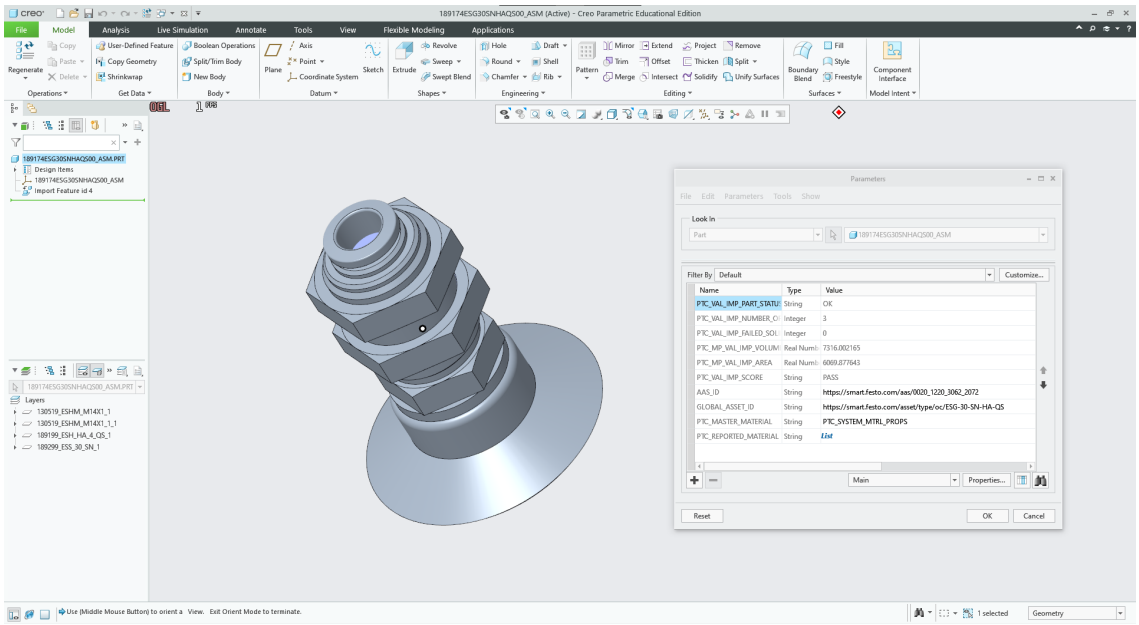
3. **Select the Target AAS**: In the Connection View, select the Asset Administration Shell you want to link or import from the provided list or dropdown.



4. **Synchronize / Import**: Click the button with the error to the right to initiate the data transfer. This will align the CAD model structure with the loaded AAS instance, or import the model files into the CAD environment.



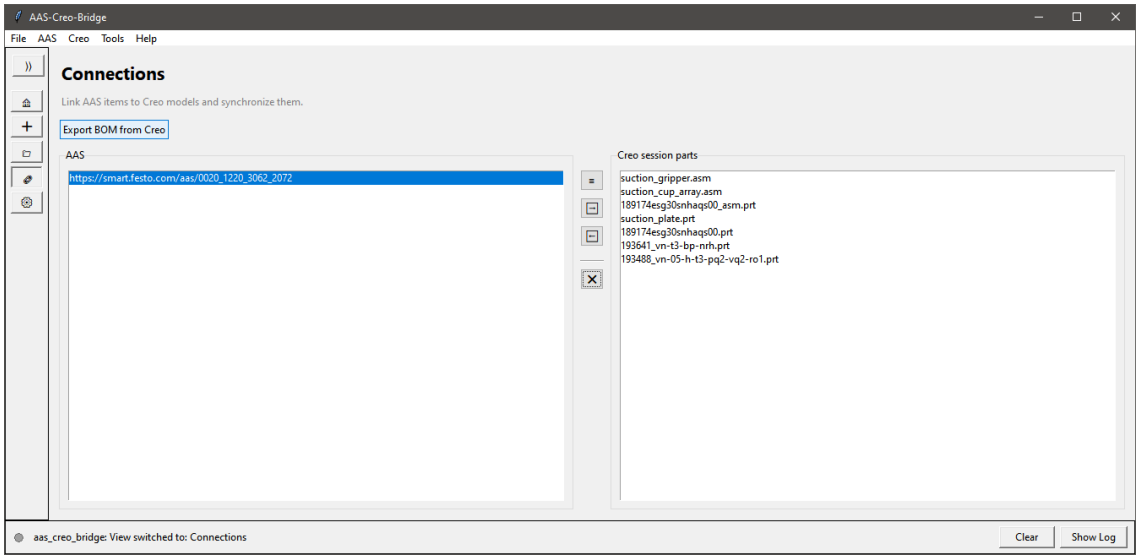
5. **Expected Changes in Creo Parametric**: After the sync completes, switch back to your Creo Parametric window. If you imported a model, new parts or assemblies should now be open. The parameters (e.g., `AAS_ID`) within your models should be set accordingly.



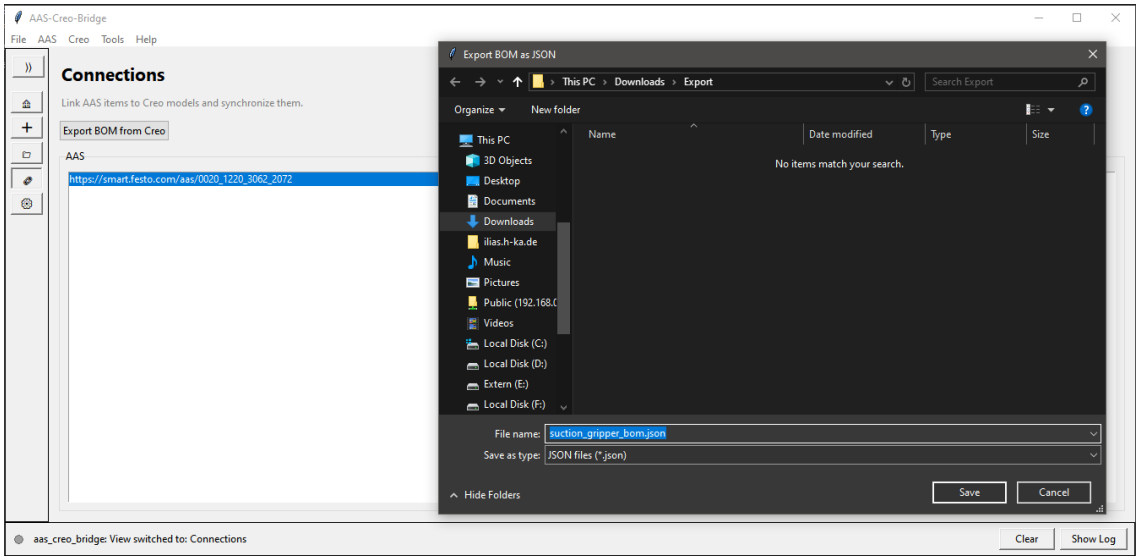
8.3.4 Exporting a Bill of Materials (BOM)

To extract a Bill of Materials from an open Creo assembly, follow these steps:

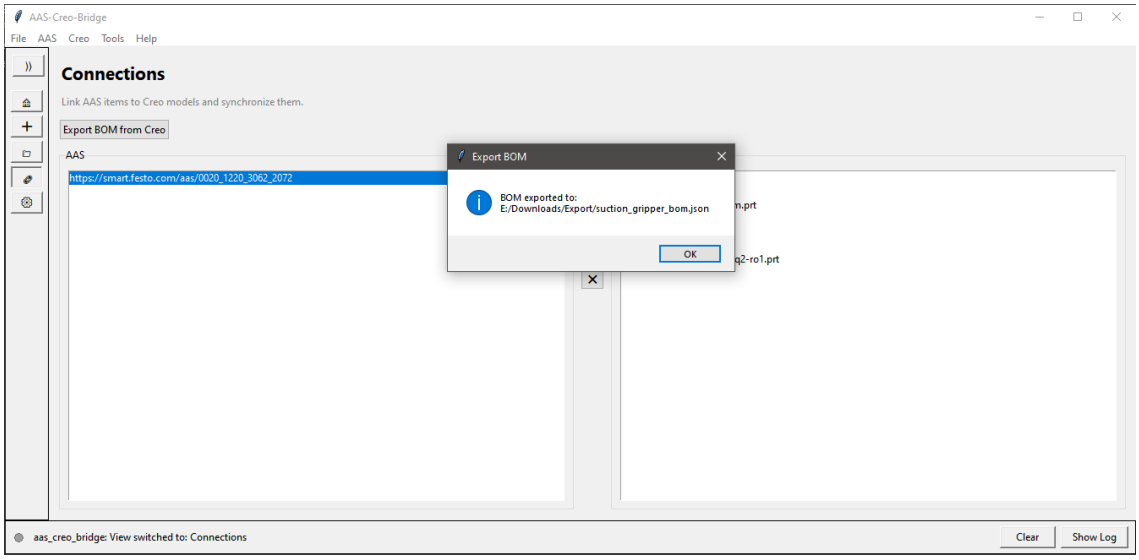
1. **Prepare Creo Parametric**: Ensure that your assembly is open and active in your PTC Creo session.
2. **Navigate to the Connection View**: Go to the **Connection** tab in the side navigation pane and verify that the Bridge is successfully connected to Creo.
3. **Start Export/Sync**: Click the **Export BOM** (or equivalent Sync) button. The Bridge will read the hierarchical structure and part information from the active Creo session.



4. **Save the BOM**: A file dialog will open to set a location for saving the extracted BOM. The BOM can only be saved as a `.json` file.



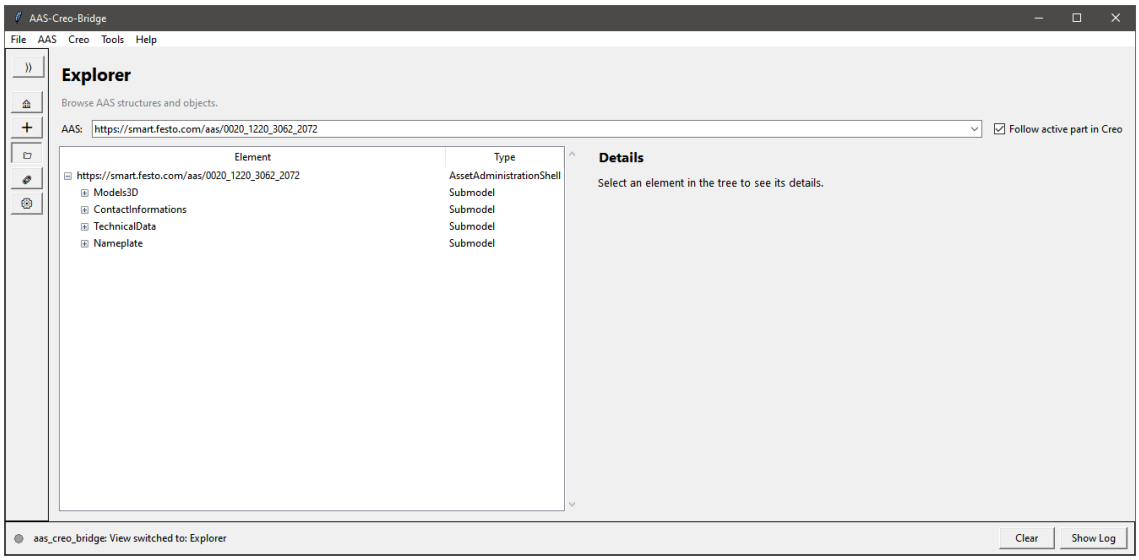
5. **Wait for the export**: Once completed, the extracted BOM will be saved to the specified location. A pop-up message will confirm the successful export.



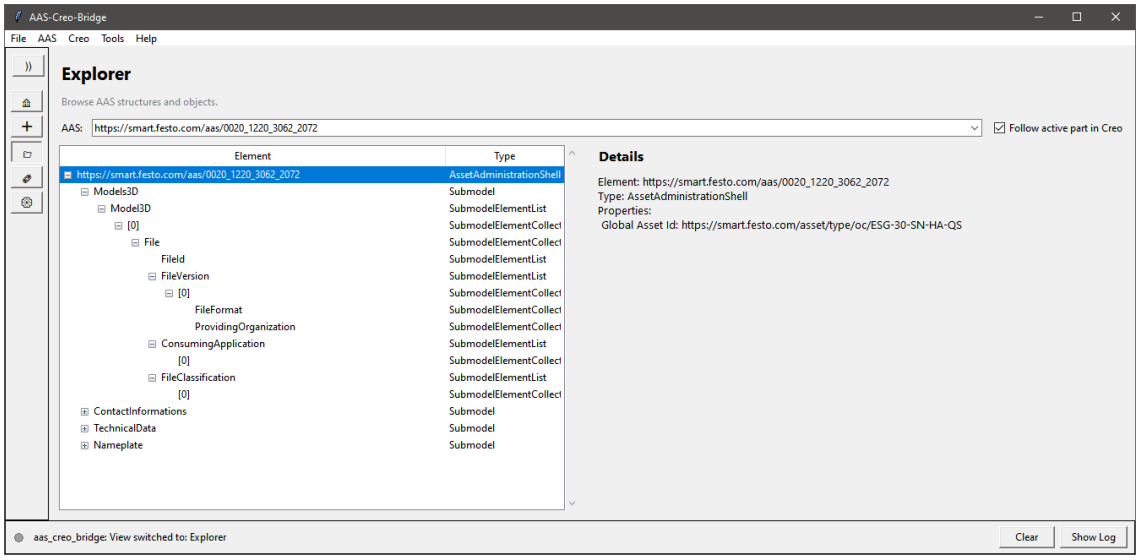
8.3.5 Using the Explorer View

The Explorer View allows you to inspect the internal structure and properties of your loaded Asset Administration Shells.

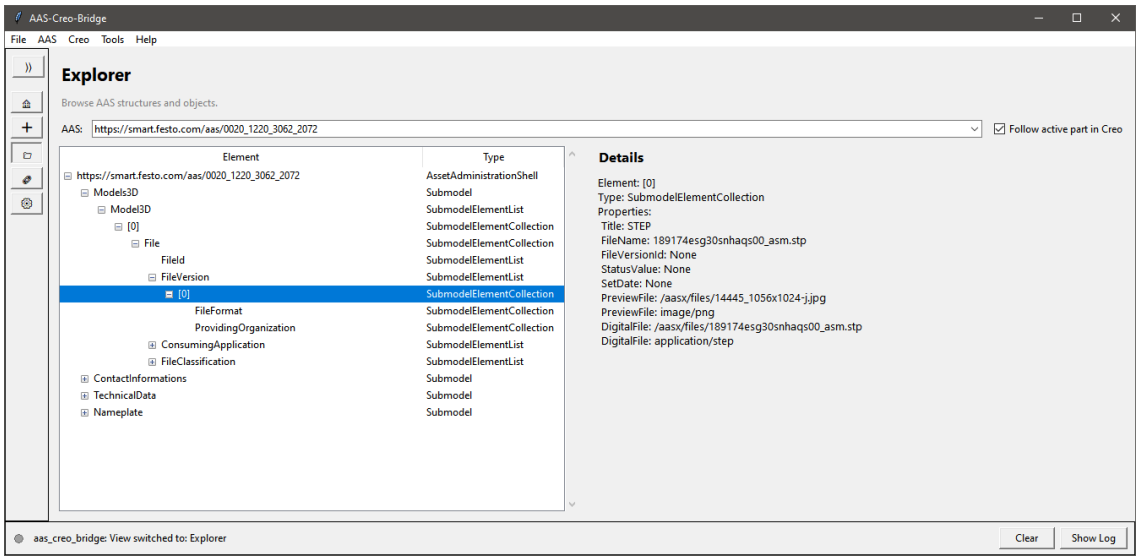
1. **Navigate to the Explorer Tab**: In the side navigation pane, click on the Folder icon to open the **Explorer** tab.



2. **Select the AAS**: At the top you can select the AAS you want to view. If you select "Follow active part in Creo" the currently active part from creo will be displayed if an AAS is linked to it. To activate a part open it so it is the top part in the model tree. The left tree panel displays a hierarchical view of the AAS's contents. Expand the Asset Administration Shell to view its Submodels and contained Submodel Elements.



3. **View Element Details**: Select any specific element (e.g., a Property, BOM entry, or Concept Description) in the tree. The detail panel will display its attributes, such as values, properties, and category information.



CHAPTER

NINE

INSTALLATION GUIDE

There are no binaries released yet. Follow the Developer Guide below to set up a development environment and run the application from source.

CHAPTER

TEN

GETTING STARTED FOR DEVELOPERS

10.1 Prerequisites

- Python 3.12
- Git
- Creo Parametric 12 with JLink enabled
- Java 21 or higher (required when not using the embedded JRE option)

10.2 1) Clone the repository

```
git clone https://github.com/MikaSchmitt/AAS-Creo-Bridge.git
cd AAS-Creo-Bridge
```

10.3 2) Create and activate a virtual environment

```
py -3.12 -m venv .venv
.\.venv\Scripts\Activate.ps1
python -m pip install --upgrade pip
```

10.4 3) Install dependencies

```
python .\scripts\setup.py --install-deps --embedded-jre
```

- `--install-deps`: install all necessary python packages
- `--embedded-jre`:This will download a version of creoson with an embedded JRE.
- `--force`: override existing creoson installation.

10.5 4) Configure IDE

Add `stubs/` to your interpreter search path to enable type hints.

10.6 5) Run the application

```
python -m src.aas_creo_bridge
```

10.7 Testing

```
pytest
pytest tests\app\test_sync_manager.py
```

10.8 Next

Take a look at the following documentation

- ***Architecture Overview*** — System design and data flow

CHAPTER

ELEVEN

AAS ADAPTER

11.1

11.1.1 Purpose

The `aas_adapter` is a toolkit for working with AAS. It builds upon the basyx-python-sdk and extends its functionality with classes and helper functions for common tasks. The `aas_adapter` is the core of the `aas_adapter` and was designed in conjunction with the `aas_creo_bridge` and therefore currently focuses on workflows related to CAD.

11.1.2 Featureset

- **AASX import:** AASX files can be loaded from disk to import AAS into the program.
- **in-memory registry:** A registry for storing and managing imported AAS and their models in memory.
- **model extraction:** Extract models files from Model3D and MCAD submodels.
- **model selection:** Select the best model from a list of models based on a given consuming application and or file format.
- **file materialization:** Materialize a model into a file based on a given file format for import into the CAD software.

11.1.3 Public API

functions:

- import_aasx,
- AASXRegistry,
- get_models_from_aas,
- select_best_model,
- materialize_model_file

data classes:

- ConsumingApplication,
- FileFormat,

- FileMetadata,
- FileData,
- Version

11.1.4 Workflow

A typical workflow for importing an AASX file could look like this:

1. Import the AASX with `import_aasx()`
2. Register the AASX `AASXRegistry.register()`
3. Get the models from the AAS with `get_models_from_aas()`
4. Select a model file based on preferred consuming application and file format with `select_best_model()`
5. Write the selected model file to disk with `materialize_model_file()`

11.1.5 Minimal usage example

The following example demonstrates extracting a model file from an AASX file and materializing it.

```
from pathlib import Path
from aas_adapter import (
    import_aasx,
    AASXRegistry,
    get_models_from_aas,
    select_best_model,
    ConsumingApplication,
    FileFormat,
    materialize_model_file,
)

### Importing an AASX file

file_path = Path("example.aasx")
result = import_aasx(file_path)

aasx_registry = AASXRegistry()
aasx_registry.register(result)

### Retrieving AASX from registry
# aasx is technically the same as result in this example,
# but we still go through the registry for demonstration purposes

aas_id = result.shells[0]
aasx = aasx_registry.get_aasx(aas_id)

models = get_models_from_aas(aasx, aas_id)
```

(continues on next page)

(continued from previous page)

```
best_model = select_best_model(
    models,
    [ConsumingApplication("Creo Parametric", "12", "CREO12")],
    [FileFormat("STEP", "AP214", "STEP-214")]
)

output_path = Path("")
prepared_model = materialize_model_file(aasx, best_model, output_path)
```

11.1.6 Known limitations

`ExternalFile` in `Model3D` is detected but the extraction is not supported yet. Occurrences of `ExternalFile` are logged.

The file format in `Model3D` and `MCAD` is not standardized in the specification. The `aas_adapter` assumes that the file format is present as in the file format specification by CADENAS.

11.1.7 Related Documentation

- *Getting Started* - How to set up the development environment and run the application.
- basyx-python-sdk - The core library of the `aas_adapter`.
- *Project architecture* - get an overview over the project's architecture
- *AAS-Adapter Importer* - How to import AASX files.
- *AAS-Adapter Registry* - How to manage the registry of imported AAS.
- *AAS-Adapter Extractor* - How to extract 3D models from imported shells.
- *AAS-Adapter Selection* - How to select the best model from a list of models.
- *AAS-Adapter Materializer* - How to materialize models into files.

11.2 Importer

Path: `src/aas_adapter/importer.py`

11.2.1 Overview

The importer module is the entry point for loading AASX (Asset Administration Shell eXchange) files into the application. It reads an AASX package (which is a ZIP-based format), extracts and parses its contents using the BaSyx AAS SDK, and returns a result containing the AAS object store, supplementary files, package metadata, and a list of discovered Asset Administration Shell (AAS) identifiers.

It requires a `Path` to an existing `.aasx` file. Given that path, it creates a BaSyx object store and file store, reads package content via `AASXReader`, then collects metadata, thumbnail, and AAS shell IDs. The result is returned as one immutable `AASXImportResult` object.

In short, this module does four things:

- Validates that the path exists and points to a file.
- Reads AAS objects and supplementary file references from the package.
- Discovers which identifiers are Asset Administration Shells.
- Returns all import data in a single structured object.

11.2.2 Key Concepts

AASX Format

An AASX file is a ZIP-compressed archive that conforms to the OPC (Open Packaging Convention) standard. It contains:

- XML-serialized AAS model data (objects, submodels, collections, properties)
- Supplementary files (binary model files, images, documents)
- Package-level metadata (author, creation date, description)
- An optional thumbnail image

Object Store and File Store

- `DictObjectStore` holds parsed AAS identifiables (shells, submodels, etc.).
- `DictSupplementaryFileContainer` tracks supplementary files contained in the package.

Shell Discovery

`import_aasx()` receives a set of identifiers from `reader.read_into(...)` and then checks each identifier in the object store. Only objects that are `AssetAdministrationShell` instances are returned in `result.shells`.

11.2.3 Main Components

`import_aasx(path: Path) -> AASXImportResult`

Reads an AASX package and returns a snapshot of the imported content.

Parameter:

- `path`: Path to the AASX file.

Returns:

- `AASXImportResult.path`: Source file path.
- `AASXImportResult.object_store`: Parsed identifiables.
- `AASXImportResult.file_store`: Supplementary file container.
- `AASXImportResult.metadata`: OPC core properties or `None`.

- `AASXImportResult.thumbnail`: Thumbnail bytes or `None`.
- `AASXImportResult.shells`: IDs of discovered AAS shells.

Exceptions:

- `FileNotFoundError` when `path` does not exist or is not a file.
- `ValueError` when reading/parsing fails (reader errors are wrapped and logged).

Usage Example:

```
from pathlib import Path

from aas_adapter import import_aasx

result = import_aasx(Path("model.aasx"))

for shell_id in result.shells:
    aas = result.object_store.get_identifiable(shell_id)
    print(f"Found shell: {aas.id}")

if result.metadata:
    print(f"Creator: {result.metadata.creator}")
```

`AASXImportResult` (frozen dataclass)

`AASXImportResult` is immutable after creation. This helps avoid accidental changes after registration in `AASXRegistry`.

Fields:

- `path: Path`
- `object_store: DictObjectStore`
- `file_store: DictSupplementaryFileContainer`
- `metadata: OPCCoreProperties | None`
- `thumbnail: bytes | None`
- `shells: list[str]`

11.2.4 Integration Points

Registry

Usually, import is followed by registration:

```
from pathlib import Path

from aas_adapter import AASXRegistry, import_aasx

registry = AASXRegistry()
```

(continues on next page)

(continued from previous page)

```
result = import_aasx(Path("model.aasx"))
registry.register(result)
```

Extractor

The extractor expects an import result plus a shell ID:

```
from pathlib import Path

from aas_adapter import get_models_from_aas, import_aasx

result = import_aasx(Path("model.aasx"))
if result.shells:
    models = get_models_from_aas(result, result.shells[0])
```

Dependencies

- `basyx.aas` for AASX reading and in-memory stores.
- `pyecma376_2` for OPC core properties.

11.2.5 Error Handling

Current behavior in `import_aasx()`:

Exception	When it happens	Notes
`FileNotFoundError`	`path` missing or not a file	Raised before opening reader
`ValueError`	Any exception while reading/parsing	Original error is logged and chained

The module logs failed imports with file path and traceback before raising `ValueError`.

11.2.6 Testing

Tests are in `tests/aasx_adapter/test_importer.py`.

Covered scenarios include:

- Missing file path and directory path (`FileNotFoundError`).
- Successful import wiring (stores, metadata, thumbnail, shell discovery integration).
- Reader exceptions wrapped as `ValueError` and logged.
- `_discover_shells()` behavior (empty input and filtering by shell type).

Run tests:

```
pytest tests/aasx_adapter/test_importer.py -v
```

11.2.7 Common Patterns and Gotchas

1) `shells` can be empty

Not every AASX contains `AssetAdministrationShell` objects.

2) `metadata` and `thumbnail` can be `None`

These package fields are optional.

3) `file_store` contains references, not eagerly loaded file bytes

Read supplementary files explicitly when needed.

4) `AASXImportResult` is frozen, but contained stores are still mutable objects

The dataclass fields cannot be reassigned. The store objects themselves are not deep-copied.

5) All reader/parsing errors are surfaced as `ValueError`

If callers need root-cause detail, inspect the chained exception and logs.

11.2.8 Related Documentation

- *`Extractor`*
- *`Registry`*
- BaSyx Python SDK
- ECMA-376-2 OPC

11.3 Registry

Path: `src/aas_adapter/registry.py`

11.3.1 Overview

The registry module is responsible for keeping imported AASX files available during one app session.

It requires `AASXImportResult` objects from the importer. Given such a result, it stores it in two indexes:

- by file path (`_by_path`)
- by AAS shell ID (`_by_id`)

This allows UI and sync logic to either resolve by source file or directly by shell ID.

The registry also supports listeners. A listener is just a callback without arguments that is called whenever the registry changes.

11.3.2 Key Concepts

Dual Indexing

The registry maintains two independent lookup indexes:

- **By Path** (`_by_path`): Maps file paths to AASXEntry objects. Use this when you know which AASX file was imported.
- **By Shell ID** (`_by_id`): Maps AAS shell identifiers to AASXEntry objects. Use this when you know which shell you're interested in (e.g., from the UI).

This dual structure allows flexible queries without re-searching the entire registry.

The path index is stable per file. The ID index is convenient for shell-driven workflows.

Listener Pattern

The registry follows a publish-subscribe pattern. Listeners can register callbacks to be notified when:

- A package is registered (added to the registry)
- A package is unregistered (removed from the registry)
- The entire registry is cleared

This decouples the registry from specific UI or business logic components.

Listeners

Listeners are stored in `_listeners` and called by `_notify_listeners()`. Current signature is:

```
Callable[[], None]
```

So listeners do not receive action names or payloads. They only get a "registry changed" event.

11.3.3 Main Components

`AASXRegistry.__init__()`

Creates empty path index, ID index, and listener list.

`register(result: AASXImportResult) -> None`

Adds one import result to both indexes and notifies listeners.

Flow:

1. Store result in `_by_path[result.path]`.
2. For each shell in `result.shells`, write `_by_id[shell_id]`.
3. Call `_notify_listeners()`.

If shell IDs already exist, the new value overwrites the old one in `_by_id`.

unregister(path: Path) -> None

Removes an entry from `_by_path` and notifies listeners if the path existed.

Important current behavior: it does **not** remove related entries from `_by_id`. This can leave stale ID mappings after unregistering by path.

get(key: Path | str) -> AASXImportResult | None

Returns a result by path (`Path`) or shell ID (`str`).

is_open(key: Path | str) -> bool

Checks if a given path or shell ID is currently present in the corresponding index.

list_by_path_open() -> list[AASXImportResult]

Returns all results reachable through the path index.

list_by_id_open() -> list[AASXImportResult]

Returns all results reachable through the ID index.

list_clear() -> None

Clears both indexes and notifies listeners.

add_listener(listener: Callable[[], None]) -> None

Registers a callback.

remove_listener(listener: Callable[[], None]) -> None

Removes a callback if it exists.

11.3.4 Usage Example

```
from pathlib import Path

from aas_adapter import AASXRegistry, import_aasx

registry = AASXRegistry()


def on_registry_change() -> None:
    print("Registry changed")
```

(continues on next page)

(continued from previous page)

```
registry.add_listener(on_registry_change)

result = import_aasx(Path("model.aasx"))
registry.register(result)

by_path = registry.get(result.path)
by_id = registry.get(result.shells[0]) if result.shells else None
```

11.3.5 Integration Points

Typical flow in the app:

1. Import package via `import_aasx()`.
2. Register result via `AASXRegistry.register(...)`.
3. UI reads open packages from `list_by_path_open()`.
4. Sync logic resolves selected shell IDs using `get(shell_id)`.

11.3.6 Error Handling

Scenario	Behavior
`unregister()` with unknown path	No-op, no exception
`get()` with unknown key	Returns `None`
listener callback raises	Error is logged, other listeners still run

11.3.7 Testing

The file `tests/aasx_adapter/test_registry.py` currently exists but has no active test cases.

If you extend registry behavior, recommended test cases are:

- register/get by path and by shell ID
- duplicate shell ID overwrite behavior
- unregister path behavior and current `_by_id` side effect
- listener invocation and listener exception handling
- clear behavior for both indexes

11.3.8 Common Patterns and Gotchas

1) Duplicate shell IDs overwrite old entries in `_by_id`

This is expected with current implementation and marked as a TODO in code.

2) `unregister(path)` only updates `_by_path`

After unregistering, ID lookups can still resolve stale entries.

3) Listener callback takes no arguments

If code expects action/payload arguments, it will fail at runtime.

4) Listener removal requires the same callable object

Use a named function if you need to remove it later.

5) Registry is in-memory only

No persistence is implemented. Closing the app loses state.

11.3.9 Related Documentation

- *`Importer`* — How to create `AASXImportResult` objects.
- *`Extractor`* — How to extract models from registered packages.
- *`Selection`* — Model selection from extracted results.
- *`Models`* — Data structures used by the registry.

11.4 Extractor

Path: `src/aas_adapter/extractor.py`

11.4.1 Overview

The extractor module is responsible extracting 3D Models in the form of CAD documents and related Information from AAS. The Extractor requires an AASX import result containing a object store and a file store as well as an AAS identifier. Given those it first locates `Provision of 3D Models` and `MCAD` submodels. Then it parses the metadata including file versions, file formats and consuming applications. It also extracts references to the supplemental files (only `DigitalFile`, `ExternalFile` is not supported yet). All of this information is then returned as a list of `FileData` objects for further processing.

11.4.2 Key Concepts

Semantic IDs

AAS submodels are identified by their **semantic ID**. Two Submodels are supported:

- **Models3D** (`https://admin-shell.io/idta/Models3D/1/0`): IDTA standard for 3D CAD model metadata.
- **MCAD** (`https://admin-shell.io/sandbox/idta/handover/MCAD/0/1/`): Legacy standard for backwards compatibility.

Submodel Structure (Models3D)

The following layout shows the submodel structure that the extractor expects for the `Models3D` Submodel. The specification defines additional submodel elements, however those are currently ignored.

```
Submodel (semantic_id = Models3D)
  ├─ Model3D[] (SubmodelElementList)
      ├─ Model3D[0] (SubmodelElementCollection)
          └─ File (SubmodelElementCollection)
              ├─ ConsumingApplication[] (SubmodelElementList)
              │   ├─ ApplicationName (Property)
              │   ├─ ApplicationVersion (Property)
              │   └─ ApplicationQualifier (Property)
              └─ FileVersion[] (SubmodelElementList)
                  ├─ FileVersionId (Property)
                  ├─ FileFormat (SubmodelElementCollection)
                  │   ├─ FormatName (Property)
                  │   ├─ FormatVersion (Property)
                  │   └─ FormatQualifier (Property)
                  ├─ DigitalFile (File, optional)
                  └─ ExternalFile (File, optional, not yet supported)
```

When traversing the Submodel all elements are identified and searched for by their `idShort`. Unnessessary elements are skipped. Consuming Application is treated as optional. The `FileVersion` Submodel Element Collection is essential. If it is missing a ValueError is raised.

Submodel Structure (MCAD)

The following layout shows the submodel structure that the extractor expects for the `MCAD` Submodel. For the `MCAD` submodel there is no documentation as it was never an official submodel and only served as a concept. Still some AAS providers used the MCAD submodel for distributing 3D model files. As it was the only submodel available for some time and as of writing the `Provision of 3D Models` submodel is very new the `MCAD` submodel is still widely used.

The `MCAD` submodel is based on the `Handover Documentation` Submodel however it doesn't have its own documentation. Therefore, we looked at various AAS from providers using MCAD (mainly FESTO) to reverseengineer its usage. This gave us the following structure:

```
Submodel (semantic_id = MCAD)
    ├─ Document (SubmodelElementCollection)
    │    ├─ DocumentClassification (SubmodelElementCollection)
    │    │    ├─ ClassIdentifyer (Property)
    │    │    └─ ClassificationSystem (Property)
    │    └─ DocumentVersion (SubmodelElementCollection)
    │         ├─ DocumentVersion (Property)
    │         ├─ DocumentName (MultiLanguageProperty)
    │         └─ DigitalFile (File)
```

The structure only shows submodel elements that are necessary for the extractor. It is possible that other providers of ASS have a different interpretation of the MCAD submodel.

To find the CAD documents the extractor takes the following approach.

1. Given an MCAD Submodel the extractor looks for the `Document` submodel element collections in the list of child elements.
2. Then the extractor looks for the `DocumentClassification` to check if the document is a CAD document. It checks for classification system `VDI2770:2020` and classification identifier `02-02`. Other classification systems and identifiers are not supported.
3. Then is looks for the `DocumentVersion` to extract the metadata of the document. It extracts the `DocumentVersion` property, the `DocumentName` property, which it expects to be the file format, and the `DigitalFile` property, which stores the path to the file and the MIME type.

File Data Collection

For each File collection, the extractor collects:

1. **Consuming Applications**: What software can open the model. (only Models3D)
2. **Metadata**: Different versions/representations of the same model, each with format and file reference.

File Format

The file format is expected to follow the CADENAS standard for file formats. In the case of `Models3D` it consists of Format Name, Format Version, and Format Qualifier. For `MCAD` the file format is given as a single string which needs to be parsed into the File Format. The extractor attempts to parse the file format string into the File Format based on the CADENAS standard.

Consuming Application

The Consuming Application follows the same format as the File Format: Application Name, Application Version and Application Qualifier.

11.4.3 Main Components

`get_models_from_aas(aasx: AASXImportResult, aas_id: str) -> list[FileData]`

Purpose:
Traverse a given AAS and return a list of all CAD documents with their metadata and consuming applications.

Parameters:

- `aasx` (`AASXImportResult`): The import result from `import_aasx()`.
- `aas_id` (`str`): The unique identifier of the AAS shell to extract models from.

Returns:
`list[FileData]` — A list of FileData objects, one per model/document found. Each FileData contains consuming applications and metadata (version, file path, format).

Exceptions:

- `ValueError`: Raised if a model collection contains no usable `FileVersion`.
- `KeyError`: May be raised if submodel references cannot be resolved (logged and skipped gracefully).
- Other exceptions from BaSyx model access are logged and may propagate.

Usage Example:

```
from pathlib import Path
from aas_adapter import import_aasx, get_models_from_aas

result = import_aasx(Path("model.aasx"))
if result.shells:
    models = get_models_from_aas(result, result.shells[0])
    for file_data in models:
        print(f"Consuming apps: {[app.application_name for app in file_data.
↪consuming_applications]}")
        for meta in file_data.metadata:
            print(f"  - Version {meta.file_version}: {meta.filepath}")
```

Consuming Applications Extraction

Function: `_extract_consuming_applications(file_collection) -> list[ConsumingApplication]`

Extracts consuming application metadata from a File collection. If there is no `ConsumingApplication` element or reading fails an empty list is returned.

Each consuming application includes:

- `application_name`: e.g., “Creo Parametric”, “STEP”
- `application_version`: e.g., “12”, “242”
- `application_qualifier`: e.g., “CREO12”, “AP242”

File Versions Extraction

Function: `_extract_file_versions(file_collection) -> list[FileMetadata]`

Extracts all file versions from a File collection.

Behavior:

- Returns one `FileMetadata` per readable `FileVersion` entry.
- **DigitalFile is optional**: If missing or unreadable, that version is skipped (logged as exception, not fatal).
- **ExternalFile is detected but not supported**: A warning is logged; the version is still processed if a DigitalFile is present.
- If neither DigitalFile nor ExternalFile is present or readable, that version is skipped.

11.4.4 MCAD Document Extraction

For submodels with MCAD semantic ID, the extractor:

1. Looks for elements with semantic ID `MCAD_DOCUMENT_SEMANTIC_ID`.
2. Validates document classification (VDI2770:2020, class 02-02).
3. Extracts document version, file path, and name.
4. Maps the document name to a `FileFormat` using the Cadenas format database (via `mcad_doc_name_to_cadenas_format`).
5. Logs a warning and skips documents if format mapping fails.

11.4.5 Related: Helpers

Uses `get_value()` and `check_expected_model()` from `helpers.py` for safe value extraction and type validation.

11.4.6 Error Handling

As the implementation of the submodels is subject to interpretation the extractor tries to be as graceful as possible while still being transparent about its behavior. Only if there is a critical error the extractor raises an error all other issues are logged and the elements skipped. The following table lists how the extractor treats error that might occur during the extraction process.

Exception	Scenario	Behavior
`ValueError` (No FileVersion)	A Model3D entry lacks any readable FileVersion.	Raised immediately; caller must handle.
`ValueError` (Type mismatch)	A submodel element is not the expected BaSyx type.	Logged and re-raised with context.
`KeyError` (Unresolved reference)	A submodel reference cannot be resolved in object store.	Skipped; iteration continues to next submodel.
Missing `ConsumingApplicatio`	File collection has no consuming applications.	Returns empty list; not an error.
Missing `DigitalFile`	FileVersion has no digital file reference.	Version skipped if ExternalFile also absent.
Invalid MCAD document name	Document name cannot be mapped to FileFormat.	Logged as warning; document skipped.

11.4.7 Testing

Tests for the extractor are located in `tests/aasx_adapter/test_extractor.py`.

Key test scenarios:

- Valid Models3D submodel with multiple files and versions → correct FileData list
- MCAD document extraction → correct FileFormat mapping
- Missing ConsumingApplication → empty application list (no error)
- Missing DigitalFile → version skipped
- No FileVersion found → `ValueError` raised
- Unresolved submodel reference → skipped (logged)

Run tests:

```
pytest tests/aasx_adapter/test_extractor.py -v
```

11.4.8 Common Patterns & Gotchas

1. ValueError on No FileVersion

If a Model3D entry contains no readable file version, `get_models_from_aas()` raises `ValueError("No FileVersion found")`. Always wrap extraction in a try-catch if you're tolerant of incomplete AAS data:

```
from aas_adapter import get_models_from_aas

try:
    models = get_models_from_aas(result, shell_id)
except ValueError as e:
    print(f"Incomplete AAS data: {e}")
    models = []
```

2. ExternalFile Is Not Supported

If a FileVersion contains an `ExternalFile` reference, the extractor logs a warning but continues. External files (references to URLs, external repositories, etc.) are not downloaded or handled; only DigitalFile entries are extracted.

3. Consuming Applications List May Be Empty

If a File collection has no `ConsumingApplication` elements, `file_data.consuming_applications` will be empty. Downstream selection logic should handle this gracefully (typically by accepting any model without app constraints).

4. MCAD Document Name Mapping Depends on Cadenas Cache

The `mcad_doc_name_to_cadenas_format()` function relies on fetching the Cadenas format list over HTTP, with a 1-day TTL cache. If the network is unavailable and the cache is stale or missing, format mapping fails and the document is skipped.

5. Submodel References May Not Resolve

If a submodel reference in the AAS points to a non-existent submodel, the reference resolution fails silently (logged as exception, iteration continues). This is by design to tolerate partially valid AAS data.

6. Semantic ID Comparison

Semantic ID matching uses object equality (BaSyx `ModelReference`). Ensure your test data uses the exact same semantic ID objects or construct them consistently.

11.4.9 Related Documentation

- **Importer**: *`docs/modules/aas_adapter/importer.md`* — How to load AASX files.
- **Selection**: *`docs/modules/aas_adapter/selection.md`* — How to select the best model from extracted options.
- **Materializer**: *`docs/modules/aas_adapter/materializer.md`* — How to extract and write model files to disk.
- **Helpers**: *`docs/modules/aas_adapter/helpers.md`* — Utilities for AAS element access and Cadenas format lookup.
- **Models**: *`docs/modules/aas_adapter/models.md`* — Data structures for consuming applications, file formats, and metadata.
- **AAS Submodel Templates **: Provision of 3D Models

CHAPTER

TWELVE

APP

CHAPTER

THIRTEEN

GUI

CHAPTER

FOURTEEN

CREO ADAPTER

14.1 Creo Connection

Path: `src/aas_creo_bridge/adapters/creo/creo_connection.py`

14.1.1 Overview

The Creo connection module starts the Creoson server and establishes a client connection with retry logic. It expects a folder that contains `creoson_run.bat`, launches the server in a new console, and then connects using `creopyson.Client`.

14.1.2 Main Components

`connect_to_creoson(server_folder, max_retries=5, delay=2) -> creopyson.Client | None`

Starts Creoson and attempts to connect.

Parameters:

- `server_folder` (`str | Path`): Folder containing `creoson_run.bat`.
- `max_retries` (`int`): Number of connection attempts.
- `delay` (`int`): Delay in seconds between attempts.

Returns a connected `creopyson.Client` or raises on failure. The function currently returns `None` only if the retry loop exits without a connection, which should not happen in normal flow.

14.1.3 Behavior

- Validates that `creoson_run.bat` exists before launching.
- Starts the server process with `subprocess.Popen` and `CREATE_NEW_CONSOLE`.
- Retries `client.connect()` until successful or attempts are exhausted.

14.1.4 Error Handling

Scenario	Behavior
`creoson_run.bat` missing	Raises `FileNotFoundError`
Server process fails to launch	Raises `RuntimeError`
Connection fails after all retries	Raises `RuntimeError`
Creoson connection error (`MissingKey`)	Raises `RuntimeError`

14.1.5 Testing

This function requires a running Creo + Creoson setup. It is exercised by the integration tests in *`tests/creo_adapter/test_bom_export.py`*.

14.1.6 Common Patterns and Gotchas

1) The server folder must be correct

`server_folder` must point to the directory that contains `creoson_run.bat`.

2) Creoson startup can take time

If connection fails intermittently, increase `max_retries` or `delay`.

14.1.7 Related Documentation

- *`docs/Modules/aas_creo_bridge/creo_adapter/model_import.md`*
- *`docs/Modules/aas_creo_bridge/creo_adapter/bom_export.md`*

14.2 Model Import

Path: `src/aas_creo_bridge/adapters/creo/model_import.py`

14.2.1 Overview

The model import module opens native Creo models or imports common exchange formats into Creo via Creoson. The function detects the format from the file extension, calls the appropriate Creoson API, and returns the opened Creo model name.

14.2.2 Main Components

`import_model_into_creo(client, path) -> str`

Opens or imports a model into Creo based on file extension.

Parameters:

- `client` (`creopyson.Client`): Connected client.
- `path` (`Path`): Absolute file path to the model.

Returns:

- `str`: The opened model name as returned by Creoson.

14.2.3 Behavior

- Validates that `path` exists, is absolute, and is a file.
- Native formats (`.asm`, `.prt`) are opened directly with `file_open`. The revision suffix (for example `.asm.1`) is stripped before opening.
- Exchange formats are imported via `interface_import_file`:
  - STEP (`.stp`, `.step`)
  - IGES (`.igs`, `.iges`) (not tested)
  - NEUTRAL (`.neu`) (not tested)
  - PV (`.pvz`, `.pvs`) (not tested)
- The imported model is opened and the opened file name is returned.

14.2.4 Error Handling

Scenario	Behavior
Path does not exist	Raises `FileNotFoundError`
Path is not absolute or not a file	Raises `ValueError`
Unsupported extension	Raises `ValueError`
Creoson import/open failure	Raises `RuntimeError`

14.2.5 Testing

This function is exercised by integration tests in *`tests/creo_adapter/test_bom_export.py`*.

14.2.6 Common Patterns and Gotchas

1) Revision suffixes in native files

Native files like `assembly.asm.1` are opened by stripping the trailing revision suffix. Ensure the file is in Creo's search path.

2) Exchange import always creates a Creo model

For exchange formats, Creoson creates a Creo model and returns its name. That name is then used in `file_open`.

14.2.7 Related Documentation

- *docs/Modules/aas_creo_bridge/creo_adapter/creo_connection.md*
- *docs/Modules/aas_creo_bridge/creo_adapter/bom_export.md*

14.3 Set Parameters

Path: `src/aas_creo_bridge/adapters/creo/set_parameter.py`

14.3.1 Overview

The set parameter module writes parameter values into Creo models via Creoson. It exposes a single-parameter function and a batch function for a whole part.

14.3.2 Main Components

`set_parameter(client, file_name, parameter, designate=True, no_create=None) -> None`

Sets a single parameter on a model.

Parameters:

- `client` (`creopyson.Client`): Connected client.
- `file_name` (`str`): Target Creo file name.
- `parameter` (`Parameter`): Parameter name, type, and value.
- `designate` (`bool | None`): Whether the parameter should be designated.
- `no_create` (`bool | None`): If `True`, do not create missing parameters.

`set_part_parameters(client, part, designate=True, no_create=None) -> None`

Sets all parameters listed in a `PartParameters` instance by calling `set_parameter` for each entry.

Parameters:

- `part` (`PartParameters`): Contains the file name and a list of parameters.

14.3.3 Behavior

- Validates that `file_name` and `parameter.name` are present.
- Logs success and failure with the target file and parameter.
- Batch mode iterates in order and stops on the first failure.

14.3.4 Error Handling

Scenario	Behavior
Missing file name	Raises `ValueError`
Missing parameter name	Raises `ValueError`
Empty parameter list (batch mode)	Raises `ValueError`
Creoson update fails	Raises `RuntimeError`

14.3.5 Testing

These functions are covered indirectly by the integration tests in *`tests/creo_adapter/test_bom_export.py`*.

14.3.6 Common Patterns and Gotchas

1) Parameter types are passed through

`parameter.type` is sent as-is to Creoson. Use Creo-compatible type strings.

2) `no_create=True` prevents new parameters

If the parameter does not exist in the model and `no_create=True`, the update fails and raises `RuntimeError`.

14.3.7 Related Documentation

- *docs/Modules/aas_creo_bridge/creo_adapter/types.md*

14.4 BOM Export

Path: `src/aas_creo_bridge/adapters/creo/bom_export.py`

14.4.1 Overview

The BOM export module fetches assembly structure data from Creo via Creoson and maps it into a structured in-memory tree. The main entry point, `get_assembly_data`, returns a `CreoBom` with a root node and an index by `seq_path`. Optional flags allow enrichment with parameters, mass properties, bounding boxes, and transform matrices.

Two helper functions return the unique component file names of an assembly for lightweight workflows.

14.4.2 Key Concepts

Seq Path

Each BOM entity is identified by a `seq_path` (for example `root.1.2`). This path is used to build the hierarchy and to index nodes for fast lookup.

Root Resolution

Creoson can return the root node either as `children` (a dict) or as the BOM object itself. The adapter normalizes both shapes and logs when data is unexpected.

Optional Enrichment

`include_parameters`, `include_mass_props`, and `include_bounding_box` trigger additional Creoson API calls per file. Results are cached per file name to avoid repeated calls when the same component appears multiple times.

14.4.3 Main Components

`get_assembly_data(...) -> CreoBom`

Fetches the BOM from Creoson, builds a tree of `CreoEntity` nodes, and optionally enriches each node.

Key parameters:

- `client`: Connected `creopyson.Client`.
- `file_`: Target assembly file name. If `None`, Creo uses the active model.
- `get_transforms`: If `True`, include `transform_matrix` on each node.

- `include_parameters`, `include_mass_props`, `include_bounding_box`: Enable enrichment from Creoson API calls.

Returns a `CreoBom` with `root` and `index`.

`get_assembly_component_files(client, target_model) -> set[str]`

Returns unique component file names for the given assembly. This is a thin wrapper around `get_assembly_component_file_names` with `include_root=False`.

`get_assembly_component_file_names(client, target_model, include_root=False) -> set[str]`

Returns unique component file names. When `include_root=True`, the root assembly file is included.

14.4.4 Error Handling

Scenario	Behavior
BOM request fails	Raises `RuntimeError`
BOM response is not a dict	Raises `RuntimeError`
Parameter/mass/bbox API call fails	Logs warning, uses empty defaults
Missing `seq_path` or `transform`	Logs warning, generates fallback path

14.4.5 Testing

Integration tests are in *`tests/creo_adapter/test_bom_export.py`*.

Run:

```
pytest tests/creo_adapter/test_bom_export.py -v
```

14.4.6 Common Patterns and Gotchas

1) `file_` is optional

When `file_` is `None`, Creo uses the active model. This can be convenient in interactive sessions but is less deterministic in automated runs.

2) Transforms are only present when requested

If `get_transforms=False`, `transform_matrix` will be `None` for all entities.

3) Cache is per file name

If the same component appears multiple times, enrichment results are reused.

14.4.7 Related Documentation

- *docs/Modules/aas_creo_bridge/creo_adapter/types.md*
- *docs/Modules/aas_creo_bridge/creo_adapter/creo_connection.md*
- *docs/Modules/aas_creo_bridge/creo_adapter/model_import.md*

14.5 Types

Path: `src/aas_creo_bridge/adapters/creo/types.py`

14.5.1 Overview

The types module defines the data structures used by the Creo adapter. It includes BOM entities and parameters as well as session tracking structures.

14.5.2 Main Components

`Parameter`

Represents a single Creo parameter.

Fields:

- `name` (`str`)
- `type` (`str`)
- `value` (`Any`)

`PartParameters`

Groups a file name with a list of parameters.

Fields:

- `file_name` (`str`)
- `parameters` (`list[Parameter]`)

CreoEntity

Represents a node in the assembly hierarchy.

Fields:

- `file_name` (`str`)
- `seq_path` (`str`)
- `level` (`int`)
- `parameters` (`list[Parameter]`)
- `mass` (`float`)
- `volume` (`float`)
- `length` (`float`)
- `width` (`float`)
- `height` (`float`)
- `transform_matrix` (`Optional[List[float]]`)
- `children` (`list[CreoEntity]`)

Methods:

- `add_child(child)`: Appends a child entity.
- `find_by_path(path)`: Recursively searches by `seq_path`.

CreoBom

Root container for the hierarchy and its index.

Fields:

- `root` (`CreoEntity`)
- `index` (`dict[str, CreoEntity]`)

Method:

- `get(path)`: Returns an entity by `seq_path` from the index.

CreoSessionFile

Immutable representation of a file in the Creo session.

Fields:

- `file_name` (`str`)
- `revision` (`int`)

`CreoSessionState`

Snapshot of the current Creo session.

Fields:

- `files` (`dict[str, CreoSessionFile]`)
- `active_file_name` (`Optional[str]`)
- `captured_at` (`datetime`, UTC)

`CreoSessionDelta`

Immutable description of changes between two session states.

Fields:

- `added` (`tuple[CreoSessionFile, ...]`)
- `removed` (`tuple[CreoSessionFile, ...]`)
- `revision_changed` (`tuple[CreoSessionFile, ...]`)
- `active_file_changed` (`bool`)
- `previous_active_file_name` (`Optional[str]`)
- `current_active_file_name` (`Optional[str]`)

Property:

- `has_changes`: `True` if any change is present.

14.5.3 Related Documentation

- *docs/Modules/aas_creo_bridge/creo_adapter/bom_export.md*
- *docs/Modules/aas_creo_bridge/creo_adapter/set_parameter.md*

CHAPTER

FIFTEEN

NEXT STEPS

15.1 Open Code Items

Several areas are still implemented as placeholders and need to be completed. This includes the GUI menu actions (Open/Save/Export/Validate/Connect/Disconnect/Sync) and some of the synchronization and linking logic in the `Connections` view. These items block a stable end-to-end workflow and should be prioritized.

15.2 Functional Extensions

To make accessing AASs easier the option to connect to a Repository could be added to the Import View. Placeholders for this functionality already exist.

At the moment, BOMs are only fetched from Creo as a structure (for example a JSON/dict from Creoson). The next step is to write this data into the AAS and model it as appropriate submodels. A second core function should be the direct import of CAD models from Creo into an AAS, including storing the files in the AASX package and referencing them in the relevant submodels.

CHAPTER

SIXTEEN

INDICES AND TABLES

- genindex
- modindex
- search